



Clase Thread



La clase Thread es una clase de Java utilizada para representar hilos.

Explicación

Thread es la clase de Java que representa hilos de ejecución. Un objeto `Thread` es simplemente un objeto. Un hilo real aparece solo cuando invocamos `start()`.

La dualidad **objeto vs hilo** es clave en la asignatura:

⚠️ Hasta que no llamas a `start()`, NO hay concurrencia.

Thread como objeto

Cuando instanciamos un `Thread`, lo que tenemos es:

- un objeto Java completamente normal,
- en estado **NEW** (nuevo),
- que todavía **no ejecuta nada**.

```
class MiHilo extends Thread {  
    @Override  
    public void run() {  
    }  
}  
  
MiHilo h = new MiHilo();
```

Aquí **no existe hilo en ejecución**.

No hay concurrencia, no hay planificación, no hay nada dinámico. Tan solo tenemos una **ENTIDAD ESTÁTICA**.

Thread como hilo

La magia ocurre aquí:

```
h.start();
```

`start()` :

- cambia el estado del hilo de **NEW** → **RUNNABLE** (listo),
- registra el hilo en la JVM,
- lo entrega al planificador del sistema operativo,
- y este ejecutará su `run()` cuando toque.

```
MiHilo h = new MiHilo();  
h.start(); // ahora SÍ existe un hilo real
```

Esto ocurre porque un hilo puede **tener diferentes estados** (para profundizar ver los apuntes sobre ciclo de vida de un hilo). y cuando nosotros lo creamos con el operador "new", lo que tenemos es un objeto de la clase Thread en estado "Nuevo".

Sin embargo, al utilizar el método `.start()`, pasa a estado "Listo", siendo elegible por el procesador, y convirtiéndose así en esa "secuencia de control independiente en ejecución" que entendemos como un hilo.

Sigue siendo un objeto de la clase `Thread`, pero su comportamiento es totalmente diferente, ya que al haberlo cambiado de estado, el planificador lo puede tomar en cuenta, y su código puede ser ejecutado de forma concurrente.

Es muy importante recordar que esto no se hace solo, salvo en los ejecutores, por lo que, **siempre que no hayamos lanzado el método `.start()`** tendremos una **ENTIDAD ESTÁTICAS**.

Si no lo lanzamos y ejecutamos el método `.run()` del objeto, lo que tendríamos es una invocación a un método de clase como otro cualquiera que podemos hacer en Java.

Muy importante:

```
h.run(); // ❌ NO crea un hilo
// simplemente ejecuta run() en el hilo actual
```

Esto es *pregunta clásica de examen*.

Ejemplo para diferenciar ambos usos

```
class MiHilo extends Thread {
    public void run() {
        System.out.println("Ejecutando en: " + Thread.currentThread().getName());
    }
}

public class Demo {
    public static void main(String[] args) {

        System.out.println("Hilo main: " + Thread.currentThread().getName());

        MiHilo h1 = new MiHilo();
        MiHilo h2 = new MiHilo();

        System.out.println("\nLlamando a run():");
        h1.run(); // se ejecuta en main → NO hay concurrencia

        System.out.println("\nLlamando a start():");
        h2.start(); // se ejecuta en un hilo independiente
    }
}
```

Peculiaridades de la clase

La clase `Thread` no impone un límite de objeto que podemos crear (ni la JVM tampoco realmente).

Esta limitación, viene desde el sistema operativo, ya que, lo que hace Java cuando nuestro hilo pasa a "**Listo**" es asignarlo a un hilo del sistema operativo.

El límite real de hilos lo marca:

- la memoria disponible,
- las restricciones del sistema operativo,
- y la configuración de la JVM.
- Si creas demasiados hilos, puedes agotar recursos → de ahí la necesidad del patrón de pool de threads (para profundizar ir a los apuntes de pool de threads y ejecutores).

Esta creación de hilos es **MANUAL Y DINÁMICA**:

- **Manual.** Los hilos no se crean solos, sino que somos nosotros los que tenemos que escribir código para definirlos y lanzarlos.
- **Dinámico.** Ya que utilizando cosas como los ejecutores podemos crear y destruir hilos durante la ejecución de nuestro programa.

Prioridad

Internamente los hilos tienen una prioridad.

No es muy importante porque el planificador no la tomará muy en cuenta. Si se quiere profundizar revisar los apuntes sobre prioridades en hilos.

Lo único que debemos saber es que, por defecto es 5, y no podemos tomar nada como seguro si hablamos de prioridades, ya que en Java no podemos planificar o definir prioridades por el carácter generalista del lenguaje.

Métodos de la clase

1. `start()`

- Cambia el estado del hilo a *listo para ejecutarse*.
- **No bloquea.**
- El código de `run()` se ejecutará cuando el planificador lo decida.

2. `run()`

El cuerpo del hilo.

Regla de oro:

Nunca llames a `run()` directamente si lo que quieres es concurrencia.

3. `join()`

Bloquea al hilo llamador hasta que el otro hilo termina.

```
h.start();  
h.join(); // el main espera a que h termine
```

Muy útil para sincronizar fases.

4. Otros métodos relevantes

- `sleep(ms)` → duerme el hilo actual (nunca deberíamos intentar sincronizar durmiendo hilos).
- `yield()` → da una pista al planificador (no garantiza nada).
- `setPriority()` → casi decorativo; Java no garantiza prioridades reales.

Usando Thread con Runnable

Otra forma de tener hilos en Java es utilizando Runnable.

Con la interfaz de runnable definimos la unidad de trabajo que le damos a nuestros objetos de la clase Thread (para saber más, revisar los apuntes sobre runnable).

Esta opción es preferible a nivel de implementación porque Java no tiene herencia múltiple.

```
class MiTarea implements Runnable {  
    @Override  
    public void run() {  
        System.out.println("Ejecutando tarea");  
    }  
}
```

```
Thread t = new Thread(new MiTarea());  
t.start();
```

Con lambda

```
Thread t = new Thread(() ->  
    System.out.println("Hola desde un hilo")  
);  
t.start();
```

En entornos más avanzados podemos usar `Callable` y `Future` junto con `ExecutorService`, pero eso entra más en el terreno de ejecutores y pools de hilos.